# Fake Audio Detection Using LSTM

Bivas Nandan Debnath Umme Hasnine Nusrat Nafisa
Department of Computer Science and Engineering
University of Asia Pacific, Dhaka, Bangladesh
22201256@uap-bd.edu, 22201269@uap-bd.edu, 22201253@uap-bd.edu

***Abstract*—The rapid development of deepfake audio detection is an imperative challenge due to the proliferation of AI-generated synthetic speech and the creation of highly realistic fake audio that can cause serious threats to digital trust, voice authentication systems, and media integrity. In this paper, we systematically compare three deep learning architectures, namely Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN) and Gated Recurrent Unit (GRU), for binary classification of real vs. fake speech. All three models take as input Mel-frequency cepstral coefficients (MFCC) features calculated from each of the 16,792 audio samples in the Kaggle dataset. Data augmentation is used for improving the model generalization. The experimental results indicate that all three models can achieve high accuracy, in which CNN achieves 98.05%, LSTM achieves 98.91% and GRU achieves 98.62%. The results show that both recurrent and convolutional architectures are suitable for MFCC-based deepfake audio detection, with LSTM showing the best overall performance on the task.**

***Index Terms*—Deepfake Audio Detection, Fake Speech, MFCC, LSTM, CNN, GRU, Deep Learning, Binary Classification**

## I. Introduction

In recent years, artificial intelligence has greatly improved the speech synthesis systems. Modern voice cloning and text-to-speech technologies can produce speech that sounds remarkably like real humans. These technologies have useful applications in entertainment, accessibility and virtual assistants but can also be misused for fraud, misinformation, impersonation and cybercrime. As these tools become more readily available, the problem of automatic synthetic speech detection has evolved from a niche research problem to a real security concern with real world consequences.

Deepfake audio is artificially generated or manipulated human speech to sound like the voice of a real person. Detecting such fake audio is an increasingly important area of research at the intersection of machine learning and cyber security. Modern synthesis systems are constantly improving in quality, making it difficult for traditional audio analysis methods to detect the subtle artificial patterns in synthetically produced speech. Deep learning techniques have therefore gained traction in this space since they can automatically learn both temporal and spectral patterns directly from raw audio representations, without relying on hand-crafted detection rules.

This work develops a deepfake audio detection system using LSTM networks as the core architecture and implements CNN and GRU models alongside for performance comparison. The three models are trained on Mel-frequency cepstral coefficient (MFCC) features extracted from a dataset of 16,792 real and fake speech samples. The aim of this paper is to assess the efficiency of the recurrent and convolutional neural networks in classifying the real and fake speech and to find out which architecture is the most suitable in the terms of accuracy and applicability for this task.

## II. Related Work

Research into deepfake audio detection and synthetic speech analysis has increased significantly in the past decade, mainly due to the rapid progress in text-to-speech and voice conversion technologies . Early work in this space focused on classical signal processing features and statistical models. The ASVspoof challenge series [4] played a pivotal role in formalising the task as a benchmark problem and providing standardised datasets and evaluation protocols that drew widespread attention to the vulnerability of voice systems to spoofing attacks.

Deep learning has brought significant improvements to audio classification. Long Short-Term Memory networks [5] which are able to remember information over long sequences, appeared to be a natural choice for speech-related tasks where temporal context is important. The speaker verification and spoofing detection benchmarks consistently showed LSTM-based models and other recurrent architectures outperforming traditional methods, providing a good baseline for audio forensics tasks. Muruganandham et al. [3] proposed a parallel LSTM autoencoder framework consisting of attention-enhanced recurrent learning and a dynamic residual difference encoding module based on this. Their model achieved classification accuracies of up to 97% on five benchmark datasets, by fusing multiple speech features including MFCCs, prosodic, temporal and glottal parameters.It also demonstrate the strength of LSTM-based architectures when paired with rich feature representations.

Convolutional neural networks were originally developed for image recognition and later adapted to audio classification by treating spectral representations such as MFCCs as grid-like inputs [6]. CNN-based models achieved similar results with simpler sequential processing, and their efficiency was appealing for real-time detection scenarios. Chitale et al. [1] further explored this direction, by constructing a unified hybrid architecture with CNN and LSTM layers for deepfake audio detection, revealing that joint modeling of local spectral patterns and temporal dependencies can achieve good detection performance.

Cho et al. [7] proposed the Gated Recurrent Unit, a simplified version of the LSTM with fewer parameters due to the simplified two-gate mechanism, while having similar performance on sequence modeling tasks. Several studies showed that GRU trains faster than LSTM with a small drop in accuracy. The results of this study are also in agreement with this observation. Based on this, Murty et al. [2] proposed a hybrid CNN+LSTM+GRU architecture using MFCC and other speech features for deepfake detection. The model achieved a very high test accuracy of 99.84% on the In-the-Wild dataset and also showed good cross-dataset generalization on ASVspoof 5 and WaveFake confirming that the combination of convolutional and recurrent components can lead to more robust detectors than any single architecture.

More recently, transformer-based and self-supervised architectures such as wav2vec 2.0 [8] have pushed state-of-the-art performance on deepfake audio detection, achieving very low equal error rates on established benchmarks. However, these models demand substantial computational resources both for fine-tuning and inference, which limits their practicality in resource-constrained settings. MFCC features remain a widely used and effective representation for deepfake detection [9], capturing the spectral envelope of speech in a compact form that generalizes well across different synthesis systems.

Altogether, the literature suggests that the relationship between model complexity and detection performance is not monotonic for deepfake audio classification. Well-designed combinations of MFCC features with recurrent and convolutional architectures offer a practical and competitive middle ground between classical signal processing and large pretrained models. This observation motivates a careful, efficiency-aware comparison of LSTM, CNN, and GRU, which this work aims to provide.

## III. Dataset and Preprocessing

### A. Dataset Description

The dataset used in this study was obtained from Kaggle and consists of 16,792 audio samples in WAV format, divided into two classes: 9,609 real speech recordings and 7,183 fake (synthetically generated) speech samples. The dataset is moderately imbalanced, with real samples comprising approximately 57% of the total. No additional balancing techniques were applied prior to augmentation, as the augmentation step itself helped address the skew in class distribution.

TABLE I: Dataset Summary

| Attribute | Value |
|---|---|
| Dataset Source | Kaggle |
| Total Samples | 16,792 |
| Real Audio Samples | 9,609 |
| Fake Audio Samples | 7,183 |
| Audio Format | WAV |
| Task Type | Binary Classification |
| Train / Test Split | 80% / 20% |

### B. Feature Extraction

Each audio file was loaded at its original sampling rate using the `librosa` library [11]. Mel-frequency cepstral coefficients (MFCCs) were extracted as the primary feature representation, with 13 MFCC coefficients computed per frame. MFCCs are widely used in speech and audio analysis because they compactly capture the spectral envelope of the speech signal in a perceptually motivated frequency scale, making them effective for distinguishing between natural and synthetic speech patterns [9]. The resulting feature matrices were transposed to yield sequences of shape $(\text{frames} \times 13)$, representing temporal sequences of spectral snapshots.

### C. Data Augmentation

To increase the effective size of the training set and improve model generalization, a scaling-based augmentation strategy was applied. For each original sample, two additional augmented copies were generated by multiplying the MFCC feature matrix by scaling factors of 1.05 and 0.95 respectively, simulating minor amplitude variation. This tripled the dataset size from 16,792 to 50,376 samples before splitting.

### D. Sequence Padding and Splitting

Since audio samples vary in duration, the resulting MFCC sequences differ in length across samples. All sequences were padded with zeros to a uniform maximum length of 781 frames using post-padding, producing a final input shape of $(50{,}376 \times 781 \times 13)$. The dataset was then split into 80% training (40,300 samples) and 20% testing (10,076 samples) using a fixed random seed of 42 for reproducibility. Labels were one-hot encoded into two categories representing the real and fake classes.

TABLE II: Preprocessing Configuration

| Parameter | Value |
|---|---|
| MFCC Coefficients | 13 |
| Maximum Sequence Length | 781 frames |
| Padding Strategy | Post-padding |
| Augmentation Factors | 1.05 and 0.95 |
| Total Samples after Augmentation | 50,376 |
| Train / Test Split | 80% / 20% |
| Random Seed | 42 |

## IV. Methodology

### A. LSTM Model

The Long Short-Term Memory model consists of two stacked LSTM layers followed by fully connected layers. The first LSTM layer contains 64 units and returns the full sequence, enabling the second LSTM layer of 32 units to further process the temporal context. Each recurrent layer is followed by a Dropout layer with a rate of 0.2 to reduce overfitting. The output of the second LSTM layer is passed through a Dense layer of 16 units with ReLU activation, followed by a final Dense layer of 2 units with softmax activation for binary classification. A Masking layer is applied at the input to ignore zero-padded time steps [5]. The model is

compiled with the Adam optimizer at a learning rate of 0.001 and categorical cross-entropy loss.

### B. CNN Model

The model, a Convolutional Neural Network, treats the MFCC sequence as a one-dimensional signal and uses two consecutive convolutional blocks. The first block has a `Conv1D` layer with 64 filters and kernel size 3, then a `MaxPooling1D` of pool size 2 and a Dropout layer of rate 0.2. The second block uses a `Conv1D` layer with 128 filters, with the same set-up for the pooling and dropout. The output is flattened and passed through a Dense layer of 64 units with ReLU activation, another Dropout layer of rate 0.2 and a final softmax output layer [6]. The architecture takes advantage of local temporal structure in the MFCC representation, without the need to process sequences in order over the full sequence length.

### C. GRU Model

The Gated Recurrent Unit model is similar to the LSTM architecture, but the LSTM cells are replaced by the GRU cells [7]. We use two stacked GRU layers of 64 and 32 units respectively, each followed by a Dropout layer of rate 0.2. The last layers are a Dense layer with 16 units and ReLU activation and a softmax output layer with 2 units. The simplified gating mechanism of reset and update gates of GRU results in faster training in general. GRU has similar sequential modeling ability as LSTM with fewer parameters.

TABLE III: Shared Model Hyperparameters

| Parameter | Value |
|---|---|
| First Recurrent Layer Units | 64 |
| Second Recurrent Layer Units | 32 |
| Dropout Rate | 0.2 |
| Dense Layer Units | 16 |
| Output Units | 2 |
| Optimizer | Adam |
| Learning Rate | 0.001 |
| Loss Function | Categorical Cross-entropy |
| Epochs | 10 |
| Batch Size | 32 |

### D. Training Configuration

All three models were trained for 10 epochs with batch size of 32. A validation split of 20% of the training data was used to monitor generalization during training. We used the Adam optimizer with a learning rate of 0.001 and a categorical cross-entropy loss function for all models. All experiments were done in Python using TensorFlow and Keras on Google Colab.

## V. Experimental Setup

All the experiments were done using Python in Google Colab. The data set was split into 80 % training and 20 % testing, with `random_state=42` for reproducibility. The same random seed was used consistently across all three models to ensure a fair comparison.

The performance was quantified in terms of accuracy, precision, recall and F1-score obtained from the classification report. We used the `time` module in Python to record the training and validation accuracy and loss for all epochs of each model. For consistency, timing was performed in the same Colab session on the same hardware.

TABLE IV: Tools and Frameworks

| Tool / Framework | Purpose |
|---|---|
| Python 3 | Programming Language |
| NumPy / Pandas | Data Handling |
| librosa | Audio Loading and MFCC Extraction |
| TensorFlow / Keras | Model Construction and Training |
| Scikit-learn | Evaluation Metrics |
| Matplotlib / Seaborn | Visualization |

## VI. Results and Discussion

### A. LSTM Training Performance

Figure 1 shows the LSTM model's accuracy and loss curves during training and validation for 10 epochs. The model converges steadily, with training accuracy increasing from 91% at epoch 0 to around 98.5% by the last epoch. The validation accuracy starts at a higher value of around 95% and levels off at about 98.8%, showing that the model generalizes well to unseen data without substantial overfitting. The loss curves show a steady downward trend for both training and validation, further confirming the stable and reliable learning throughout the training process.

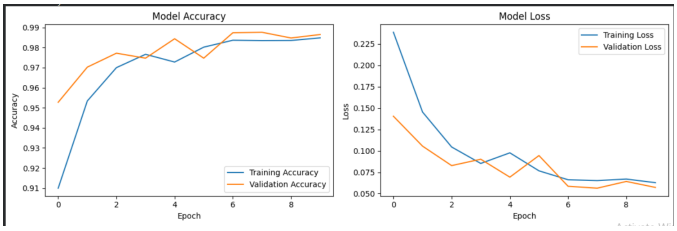



Fig. 1: LSTM model training and validation accuracy and loss over 10 epochs.

### B. Model Performance Comparison

Table V summarizes the test accuracy, precision, recall, and F1-score of all three models. LSTM achieves the highest overall accuracy at 98.91%, followed closely by GRU at 98.62% and CNN at 98.05%. The differences between models are narrow, suggesting that all three architectures are capable of learning discriminative representations from MFCC features for this task.

TABLE V: Model Performance Comparison

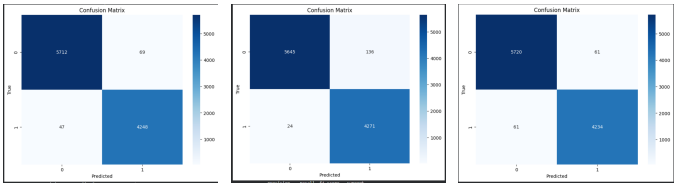
| Model | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| LSTM | 98.91% | 0.99 | 0.99 | 0.99 |
| CNN | 98.05% | 0.98 | 0.98 | 0.98 |
| GRU | 98.62% | 0.99 | 0.99 | 0.99 |

For all metrics, LSTM got the best numbers with a precision and recall of 0.99 for both classes. The model benefits from its full gating mechanism, which allows it to capture long-range temporal patterns in the MFCC sequence, a significant

advantage when the cues that differentiate real from fake speech are spread across the entire duration of the recording. The GRU model performed similarly to LSTM, but used a simpler 2-gate architecture. This result is consistent with previous studies that have shown that the simplified structure of GRU does not have a substantial impact on the performance of binary classification tasks of this kind [7].CNN delivered solid results and was notably the fastest to train among the three, making it a practical choice in scenarios where inference speed matters more than marginal accuracy gains.

### C. Confusion Matrix Analysis

Fig. 2 presents the confusion matrices for all three models on the test set. LSTM correctly classified 5712 real samples and 4248 fake samples. 69 real samples were misclassified as fake and 47 fake samples were misclassified as real. CNN successfully classified 5,645 real and 4,271 fake samples. There were 136 false negatives on the real class and 24 false negatives on the fake class. GRU classified correctly 5,720 real samples and 4,234 fake samples, with 61 false negatives and false positives. In all three models, the number of misclassifications is small and well spread between the two classes, suggesting that none of the models are heavily biased towards either real or fake predictions.




(a) LSTM (b) CNN (c) GRU

Fig. 2: Confusion matrices for LSTM, CNN, and GRU models on the test set.

### D. Summary and Key Takeaways

The results show that deep learning models trained on MFCC features are highly efficient for deepfake audio detection, with all three architectures achieving more than 98% accuracy. LSTM outperforms the other models on average by exploiting its ability to model long-range temporal dependencies across the padded speech sequences. GRU is a competitive and more efficient alternative, while CNN achieves the fastest training time and remains a strong contender in resource constrained settings. What is perhaps most interesting is how similar the three models are in their final performance, indicating that the quality of the feature representation is at least as important as the choice of architecture. [3].

## VII. Conclusion

In this work, we have performed a comparative analysis of three deep learning architectures, LSTM, CNN and GRU, for binary deepfake audio detection. MFCC features have been extracted from a dataset of 16,792 real and fake speech samples. LSTM was the best performing model with an accuracy of 98.91%, followed by GRU (98.62%) and CNN (98.05%). All three models performed well and reliably, proving that MFCC-based feature extraction is a good foundation for deepfake speech detection. These results have practical implications for the design of audio authentication systems where high accuracy and low false negative on fake audio are desirable. The architecture to choose depends on the deployment. If maximum accuracy is the priority, LSTM should be used. If a good mix between performance and efficiency is needed, then GRU should be considered. If the application requires fast inference, then CNN may be the best option. Future work could explore pre-trained audio representations such as wav2vec 2.0 or Whisper encoder features instead of hand-crafted MFCCs, bidirectional recurrent architectures, attention mechanisms and transformer-based models to see if the extra complexity yields meaningful gains on this task. Evaluating on more diverse and challenging datasets, including cross-dataset generalization to unseen synthesis systems, would also improve the practical relevance of the findings.

## References


[1] M. Chitale, A. Dhawale, M. Dubey, and S. Ghane, "A Hybrid CNN-LSTM Approach for Deepfake Audio Detection," in *Proc. 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT)*, 2024, pp. 1–6.

[2] M. Murty, S. Tomar, and S. G. Koolagudi, "A Hybrid Deep Learning Framework for Real and Deepfake Voice Detection," *Circuits, Systems, and Signal Processing*, 2026. doi: 10.1007/s00034-025-03464-4.

[3] P. Muruganandham, G. R. Thangasamy, S. Jayaraman, and R. Dharmarajan, "LSTM Autoencoder Based Parallel Architecture for Deepfake Audio Detection with Dynamic Residual Encoding and Feature Fusion," *Scientific Reports*, vol. 15, no. 1, p. 23514, 2025. doi: 10.1038/s41598-025-08198-6.

[4] T. Kinnunen et al., "ASVspoof 2017 Challenge: Assessing the Limits of Replay Spoofing Attack Detection," in *Proc. Interspeech*, 2017, pp. 2–6.

[5] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015.

[7] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.

[8] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," in *Proc. NeurIPS*, 2020, pp. 12449–12460.

[9] A. Hamza et al., "Deepfake Audio Detection via MFCC Features Using Machine Learning," *IEEE Access*, vol. 10, pp. 134018–134028, 2022.

[10] J. Yi et al., "ADD 2022: The First Audio Deep Fake Detection Challenge," in *Proc. ICASSP*, 2022, pp. 9216–9220.

[11] B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," in *Proc. 14th Python in Science Conference*, 2015, pp. 18–25.

[12] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. OSDI*, 2016, pp. 265–283.